COMPLETE PRACTICE GUIDE



SQL Complete Guide

200 Practice Questions & Solutions

From Beginner Filters to Advanced Window Functions

"200 real, scenario-based SQL problems with working query solutions — covering joins, aggregation, ranking, window functions, dates, subqueries, CTEs, string handling, and the toughest interview favorites."

TOPICS COVERED

- Filtering & Aggregation
- Duplicates & Nth-Value Problems
- Date & Time Logic
- Subqueries & Recursive CTEs
- Gaps, Islands & Pivoting
- Joins & Multi-table Queries
- Window Functions
- GROUP BY / HAVING Puzzles
- String & Pattern Matching
- Real Interview Favorites

Reference: Database Schema

All 200 questions in this guide are written against the sample database below. Keep this page open as a reference — column names used in every query map directly to these tables.

▸ employees

COLUMN	TYPE	DESCRIPTION
id	INT (PK)	Unique employee ID
name	VARCHAR	Employee full name
department_id	INT (FK)	Links to departments.department_id
manager_id	INT (FK, self)	Links to another employee's id
job_title	VARCHAR	Designation, e.g. Manager, Analyst
salary	DECIMAL	Monthly/annual salary
join_date	DATE	Date of hire
email	VARCHAR	Employee email address

▸ departments

COLUMN	TYPE	DESCRIPTION
department_id	INT (PK)	Unique department ID
department_name	VARCHAR	e.g. Engineering, Sales, HR
location	VARCHAR	City/office location

▸ customers

COLUMN	TYPE	DESCRIPTION
customer_id	INT (PK)	Unique customer ID
name	VARCHAR	Customer full name
email	VARCHAR	Customer email
phone	VARCHAR	Contact number
signup_date	DATE	Date customer registered
country	VARCHAR	Customer's country

▸ orders & order_items

TABLE	COLUMN	DESCRIPTION
orders	order_id (PK), customer_id (FK), order_date, amount	One row per order placed
order_items	order_id (FK), product_id (FK), quantity, price	Line items within an order

▸ products & categories

TABLE	COLUMN	DESCRIPTION
products	product_id (PK), product_name, category_id (FK), price, vendor_id, stock_quantity	Product catalog
categories	category_id (PK), category_name	Product category lookup

Reference: Sample Data

> Sample rows for the core tables — use these to mentally trace through any query's logic and verify what the output would look like.

▸ employees (sample rows)

ID	NAME	DEPT_ID	MANAGER_ID	JOB_TITLE	SALARY	JOIN_DATE
1	John Smith	10	NULL	Director	120000	2018-03-01
2	Aman Verma	10	1	Manager	85000	2019-06-15
3	Priya Singh	20	1	Manager	90000	2020-01-10
4	Rahul Mehta	20	3	Analyst	55000	2021-09-20
5	Sara Khan	10	2	Analyst	52000	2022-02-05

▸ departments (sample rows)

DEPARTMENT_ID	DEPARTMENT_NAME	LOCATION
10	Engineering	New York
20	Sales	Chicago
30	HR	Boston

▸ customers & orders (sample rows)

CUSTOMER_ID	NAME	ORDER_ID	ORDER_DATE	AMOUNT
101	Alice Brown	5001	2024-01-12	250.00
101	Alice Brown	5004	2024-03-02	180.50
102	Mike Lee	5002	2024-01-20	89.99
103	Wei Zhang	5003	2024-02-15	540.00

▸ products (sample rows)

PRODUCT_ID	PRODUCT_NAME	CATEGORY_ID	PRICE	STOCK_QUANTITY
201	Wireless Mouse	1	25.99	120
202	Mechanical Keyboard	1	79.99	0
203	Office Chair	2	199.00	40

Basic Filtering & Aggregation

Foundational SELECT, WHERE, GROUP BY and aggregate-function problems every SQL practitioner must master.

Q1 Find employees earning more than the company average salary.

```
SELECT *
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Q2 Count the number of employees in each department.

```
SELECT department_id, COUNT(*) AS total_employees
FROM employees
GROUP BY department_id;
```

Q3 Find employees who do not have a manager assigned.

```
SELECT *
FROM employees
WHERE manager_id IS NULL;
```

Q4 Find all employees whose email is missing (NULL).

```
SELECT *
FROM employees
WHERE email IS NULL;
```

Q5 Find employees hired in the current year.

```
SELECT *
FROM employees
```

Basic Filtering & Aggregation

Foundational SELECT, WHERE, GROUP BY and aggregate-function problems every SQL practitioner must master.

Q6 Find the top 3 highest paid employees in each department.

```
SELECT *
FROM (
  SELECT e.*,
         ROW_NUMBER() OVER (PARTITION BY department_id
ORDER BY salary DESC) AS rnk
  FROM employees e
) ranked
WHERE rnk <= 3;
```

Q7 Find employees whose name starts with 'A'.

```
SELECT *
FROM employees
WHERE name LIKE 'A%';
```

Q8 Find employees whose name ends with 'n'.

```
SELECT *
FROM employees
WHERE name LIKE '%n';
```

Q9 Find the total salary expense per department.

```
SELECT department_id, SUM(salary) AS total_salary
FROM employees
GROUP BY department_id;
```

Basic Filtering & Aggregation

Foundational SELECT, WHERE, GROUP BY and aggregate-function problems every SQL practitioner must master.

Q10 Find the highest and lowest salary in the company.

```
SELECT MAX(salary) AS highest_salary, MIN(salary) AS
lowest_salary
FROM employees;
```

Q11 Find employees who earn between 40000 and 80000.

```
SELECT *
FROM employees
WHERE salary BETWEEN 40000 AND 80000;
```

Q12 Find the number of distinct job titles in the company.

```
SELECT COUNT(DISTINCT job_title) AS distinct_titles
FROM employees;
```

Q13 Find employees whose salary is not NULL and greater than zero.

```
SELECT *
FROM employees
WHERE salary IS NOT NULL AND salary > 0;
```

Q14 Find all employees sorted by salary in descending order.

```
SELECT *
FROM employees
ORDER BY salary DESC;
```

Basic Filtering & Aggregation

Foundational SELECT, WHERE, GROUP BY and aggregate-function problems every SQL practitioner must master.

Q15 Find the second lowest salary in the employees table.

```
SELECT MIN(salary) AS second_lowest
FROM employees
WHERE salary > (SELECT MIN(salary) FROM employees);
```

Q16 Find departments having an average salary above 50000.

```
SELECT department_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY department_id
HAVING AVG(salary) > 50000;
```

Q17 Find employees whose job title is either 'Manager' or 'Director'.

```
SELECT *
FROM employees
WHERE job_title IN ('Manager', 'Director');
```

Q18 Find employees who joined between two given dates.

```
SELECT *
FROM employees
WHERE join_date BETWEEN '2023-01-01' AND '2023-12-31';
```

Q19 Count how many employees have a salary above 100000.

```
SELECT COUNT(*) AS high_earners
```

Basic Filtering & Aggregation

Foundational SELECT, WHERE, GROUP BY and aggregate-function problems every SQL practitioner must master.

Q20 Find the average salary rounded to two decimal places.

```
SELECT ROUND(AVG(salary), 2) AS avg_salary
FROM employees;
```

Q21 List unique department IDs from the employees table.

```
SELECT DISTINCT department_id
FROM employees;
```

Q22 Find employees with a salary exactly equal to the maximum salary.

```
SELECT *
FROM employees
WHERE salary = (SELECT MAX(salary) FROM employees);
```

Q23 Find the total number of employees in the company.

```
SELECT COUNT(*) AS total_employees
FROM employees;
```

Q24 Find employees whose name contains the substring 'an'.

```
SELECT *
FROM employees
WHERE name LIKE '%an%';
```

Basic Filtering & Aggregation

Foundational SELECT, WHERE, GROUP BY and aggregate-function problems every SQL practitioner must master.

Q25 Find the 5 most recently hired employees.

```
SELECT *
FROM employees
ORDER BY join_date DESC
LIMIT 5;
```

Multi-table problems testing INNER, LEFT, RIGHT and SELF joins on real-world schemas.

Q26 Find employees who are not assigned to any department.

```
SELECT e.*
FROM employees e
LEFT JOIN departments d ON e.department_id =
d.department_id
WHERE d.department_id IS NULL;
```

Q27 Find customers who have never placed an order.

```
SELECT c.*
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Q28 Find products that have never been sold.

```
SELECT p.*
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
```

Joins-Based Problems

Multi-table problems testing INNER, LEFT, RIGHT and SELF joins on real-world schemas.

Q29 Display each employee along with their manager's name.

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.id;
```

Q30 Find departments along with their employee count, including departments with zero employees.

```
SELECT d.department_name, COUNT(e.id) AS employee_count
FROM departments d
LEFT JOIN employees e ON d.department_id =
e.department_id
GROUP BY d.department_name;
```

Q31 Find students enrolled in more than 2 courses.

```
SELECT student_id, COUNT(course_id) AS total_courses
FROM enrollments
GROUP BY student_id
HAVING COUNT(course_id) > 2;
```

Q32 Find orphaned order_items rows whose product no longer exists.

```
SELECT oi.*
FROM order_items oi
LEFT JOIN products p ON oi.product_id = p.product_id
WHERE p.product_id IS NULL;
```

Joins-Based Problems

Multi-table problems testing INNER, LEFT, RIGHT and SELF joins on real-world schemas.

Q33 Find pairs of employees who work in the same department.

```
SELECT e1.name AS employee1, e2.name AS employee2,
e1.department_id
FROM employees e1
JOIN employees e2
  ON e1.department_id = e2.department_id
 AND e1.id < e2.id;
```

Q34 Find customers who placed orders in 2024 along with their total spend.

```
SELECT c.customer_id, c.name, SUM(o.amount) AS
total_spend
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
WHERE EXTRACT(YEAR FROM o.order_date) = 2024
GROUP BY c.customer_id, c.name;
```

Q35 Find products along with the category name they belong to.

```
SELECT p.product_name, c.category_name
FROM products p
JOIN categories c ON p.category_id = c.category_id;
```

Q36 Find employees who work in departments located in 'New York'.

```
SELECT e.*
```

Joins-Based Problems

Multi-table problems testing INNER, LEFT, RIGHT and SELF joins on real-world schemas.

Q37 Find the manager with the most direct reports.

```
SELECT manager_id, COUNT(*) AS direct_reports
FROM employees
WHERE manager_id IS NOT NULL
GROUP BY manager_id
ORDER BY direct_reports DESC
LIMIT 1;
```

Q38 Find all orders along with customer name and product name.

```
SELECT o.order_id, c.name AS customer, p.product_name
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id;
```

Q39 Find employees who share the same job title as 'John'.

```
SELECT e2.*
FROM employees e1
JOIN employees e2 ON e1.job_title = e2.job_title
WHERE e1.name = 'John' AND e2.name <> 'John';
```

Q40 Find departments that have at least one employee earning above 90000.

```
SELECT DISTINCT d.department_name
```

Joins-Based Problems

Multi-table problems testing INNER, LEFT, RIGHT and SELF joins on real-world schemas.

Q41 Find customers who ordered every product in a given category.

```
SELECT o.customer_id
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
WHERE p.category_id = 1
GROUP BY o.customer_id
HAVING COUNT(DISTINCT p.product_id) = (
  SELECT COUNT(*) FROM products WHERE category_id = 1
);
```

Q42 Find employees and the total number of projects each is assigned to.

```
SELECT e.name, COUNT(pa.project_id) AS total_projects
FROM employees e
LEFT JOIN project_assignments pa ON e.id =
pa.employee_id
GROUP BY e.name;
```

Q43 Find vendors who supply products that are currently out of stock.

```
SELECT DISTINCT v.vendor_name
FROM vendors v
JOIN products p ON v.vendor_id = p.vendor_id
WHERE p.stock_quantity = 0;
```

Joins-Based Problems

Multi-table problems testing INNER, LEFT, RIGHT and SELF joins on real-world schemas.

Q44 Find the department with the highest total salary expense.

```
SELECT d.department_name, SUM(e.salary) AS total_salary
FROM departments d
JOIN employees e ON d.department_id = e.department_id
GROUP BY d.department_name
ORDER BY total_salary DESC
LIMIT 1;
```

Q45 Find customers who have placed orders for products costing more than 500.

```
SELECT DISTINCT c.customer_id, c.name
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
WHERE p.price > 500;
```

Q46 Find employees who joined before their manager.

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
JOIN employees m ON e.manager_id = m.id
WHERE e.join_date < m.join_date;
```

Q47 Find all combinations of employees and departments using a cross join (for testing).

Joins-Based Problems

Multi-table problems testing INNER, LEFT, RIGHT and SELF joins on real-world schemas.

Q48 Find suppliers who supply more than one category of product.

```
SELECT vendor_id
FROM products
GROUP BY vendor_id
HAVING COUNT(DISTINCT category_id) > 1;
```

Q49 Find the employee count for each manager, including managers with zero reports.

```
SELECT m.id, m.name, COUNT(e.id) AS reports
FROM employees m
LEFT JOIN employees e ON e.manager_id = m.id
GROUP BY m.id, m.name;
```

Q50 Find customers whose total order amount exceeds 10000.

```
SELECT c.customer_id, c.name, SUM(o.amount) AS
total_spend
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name
HAVING SUM(o.amount) > 10000;
```

Joins-Based Problems

Multi-table problems testing INNER, LEFT, RIGHT and SELF joins on real-world schemas.

Q51 Find products purchased by the same customers who bought product X.

```
SELECT DISTINCT oi2.product_id
FROM order_items oi1
JOIN orders o1 ON oi1.order_id = o1.order_id
JOIN orders o2 ON o1.customer_id = o2.customer_id
JOIN order_items oi2 ON o2.order_id = oi2.order_id
WHERE oi1.product_id = 'X' AND oi2.product_id <> 'X';
```

Q52 Find employees who have never been assigned to any project.

```
SELECT e.*
FROM employees e
LEFT JOIN project_assignments pa ON e.id =
pa.employee_id
WHERE pa.employee_id IS NULL;
```

Q53 Find the average order amount per customer along with customer name.

```
SELECT c.name, AVG(o.amount) AS avg_order_amount
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.name;
```

Joins-Based Problems

Multi-table problems testing INNER, LEFT, RIGHT and SELF joins on real-world schemas.

Q54 Find departments with more managers than regular employees.

```
SELECT department_id
FROM employees
GROUP BY department_id
HAVING SUM(CASE WHEN job_title='Manager' THEN 1 ELSE 0
END) >
       SUM(CASE WHEN job_title<>'Manager' THEN 1 ELSE 0
END);
```

The most frequently asked interview pattern: finding duplicates and Nth highest/lowest values using several techniques.

Q55 Find the second highest salary from the employees table.

```
SELECT MAX(salary) AS SecondHighestSalary
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Q56 Find the third highest salary using a window function.

```
SELECT salary
FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary
DESC) AS rnk
  FROM employees
) ranked
WHERE rnk = 3
LIMIT 1;
```

Duplicates, Ranking & Nth Value

The most frequently asked interview pattern: finding duplicates and Nth highest/lowest values using several techniques.

Q57 Find the Nth highest salary using a generic, parameterised query.

```
SELECT DISTINCT salary
FROM employees e1
WHERE (N - 1) = (
  SELECT COUNT(DISTINCT salary)
  FROM employees e2
  WHERE e2.salary > e1.salary
);
```

Q58 Find duplicate records based on the employee name.

```
SELECT name, COUNT(*) AS occurrences
FROM employees
GROUP BY name
HAVING COUNT(*) > 1;
```

Q59 Delete duplicate rows keeping only one copy.

```
DELETE FROM employees
WHERE id NOT IN (
  SELECT MIN(id)
  FROM employees
  GROUP BY name, email
);
```

Duplicates, Ranking & Nth Value

The most frequently asked interview pattern: finding duplicates and Nth highest/lowest values using several techniques.

Q60 Find duplicate emails along with the number of times they occur.

```
SELECT email, COUNT(*) AS total
FROM employees
GROUP BY email
HAVING COUNT(*) > 1;
```

Q61 Rank employees by salary within each department.

```
SELECT name, department_id, salary,
       RANK() OVER (PARTITION BY department_id ORDER BY
salary DESC) AS dept_rank
FROM employees;
```

Q62 Find ties in salary ranking using DENSE_RANK.

```
SELECT name, salary,
       DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
FROM employees;
```

Q63 Find the lowest salary in each department.

```
SELECT department_id, MIN(salary) AS lowest_salary
FROM employees
GROUP BY department_id;
```

Duplicates, Ranking & Nth Value

The most frequently asked interview pattern: finding duplicates and Nth highest/lowest values using several techniques.

Q64 Find the second highest salary per department.

```
SELECT department_id, salary
FROM (
  SELECT department_id, salary,
         DENSE_RANK() OVER (PARTITION BY department_id
ORDER BY salary DESC) AS rnk
  FROM employees
) ranked
WHERE rnk = 2;
```

Q65 Find rows that are exact duplicates across all columns.

```
SELECT name, email, salary, department_id, COUNT(*)
FROM employees
GROUP BY name, email, salary, department_id
HAVING COUNT(*) > 1;
```

Q66 Find employees with a unique (non-duplicated) salary value.

```
SELECT salary
FROM employees
GROUP BY salary
HAVING COUNT(*) = 1;
```

Q67 Assign a row number to each employee ordered by hire date.

```
SELECT name, join_date
```

Duplicates, Ranking & Nth Value

The most frequently asked interview pattern: finding duplicates and Nth highest/lowest values using several techniques.

Q68 Find the top 1 highest paid employee per job title using RANK.

```
SELECT *
FROM (
  SELECT *, RANK() OVER (PARTITION BY job_title ORDER BY
salary DESC) AS rnk
  FROM employees
) ranked
WHERE rnk = 1;
```

Q69 Remove duplicate customer records keeping the most recently updated one.

```
DELETE FROM customers
WHERE customer_id NOT IN (
  SELECT customer_id FROM (
    SELECT customer_id,
           ROW_NUMBER() OVER (PARTITION BY email ORDER
BY updated_at DESC) AS rn
    FROM customers
  ) t
  WHERE rn = 1
);
```

Duplicates, Ranking & Nth Value

The most frequently asked interview pattern: finding duplicates and Nth highest/lowest values using several techniques.

Q70 Find the Nth highest order amount per customer.

```
SELECT customer_id, amount
FROM (
  SELECT customer_id, amount,
         DENSE_RANK() OVER (PARTITION BY customer_id
ORDER BY amount DESC) AS rnk
  FROM orders
) ranked
WHERE rnk = 2;
```

Q71 Find duplicate phone numbers shared by multiple customers.

```
SELECT phone, COUNT(*) AS total
FROM customers
GROUP BY phone
HAVING COUNT(*) > 1;
```

Q72 Use NTILE to split employees into 4 salary quartiles.

```
SELECT name, salary,
       NTILE(4) OVER (ORDER BY salary) AS quartile
FROM employees;
```

Duplicates, Ranking & Nth Value

The most frequently asked interview pattern: finding duplicates and Nth highest/lowest values using several techniques.

Q73 Find the second most expensive product in each category.

```
SELECT category_id, product_name, price
FROM (
  SELECT category_id, product_name, price,
         DENSE_RANK() OVER (PARTITION BY category_id
ORDER BY price DESC) AS rnk
  FROM products
) ranked
WHERE rnk = 2;
```

Q74 Find employees whose salary matches another employee's salary exactly.

```
SELECT e1.name, e1.salary
FROM employees e1
JOIN employees e2
  ON e1.salary = e2.salary AND e1.id <> e2.id;
```

Duplicates, Ranking & Nth Value

The most frequently asked interview pattern: finding duplicates and Nth highest/lowest values using several techniques.

Q75 Identify and keep only the latest order per customer, deleting older duplicates of the same product.

```
DELETE FROM orders
WHERE order_id NOT IN (
  SELECT order_id FROM (
    SELECT order_id,
           ROW_NUMBER() OVER (PARTITION BY customer_id,
product_id ORDER BY order_date DESC) AS rn
    FROM orders
  ) t
  WHERE rn = 1
);
```

Q76 Find the rank of a specific employee's salary within the company.

```
SELECT name, salary,
       RANK() OVER (ORDER BY salary DESC) AS
overall_rank
FROM employees
WHERE name = 'John';
```

Q77 Find duplicate rows in a transactions table based on amount and date.

```
SELECT amount, transaction_date, COUNT(*)
FROM transactions
GROUP BY amount, transaction_date
```

Duplicates, Ranking & Nth Value

The most frequently asked interview pattern: finding duplicates and Nth highest/lowest values using several techniques.

Q78 Find the highest salary that is not unique (shared by more than one employee).

```
SELECT salary
FROM employees
GROUP BY salary
HAVING COUNT(*) > 1
ORDER BY salary DESC
LIMIT 1;
```

Q79 Find the bottom 3 performing products by total sales using ranking.

```
SELECT product_id, total_sales
FROM (
  SELECT product_id, SUM(quantity * price) AS
total_sales,
         RANK() OVER (ORDER BY SUM(quantity * price)
ASC) AS rnk
  FROM order_items
  GROUP BY product_id
) ranked
WHERE rnk <= 3;
```

Q80 Find duplicate combinations of student and course in an enrollments table.

```
SELECT student_id, course_id, COUNT(*)
FROM enrollments
```

Duplicates, Ranking & Nth Value

The most frequently asked interview pattern: finding duplicates and Nth highest/lowest values using several techniques.

Q81 Find the percentile rank of each employee's salary.

```
SELECT name, salary,
       PERCENT_RANK() OVER (ORDER BY salary) AS
percentile_rank
FROM employees;
```

Q82 Find employees who have the same salary as the department's average.

```
SELECT e.*
FROM employees e
JOIN (
  SELECT department_id, AVG(salary) AS avg_salary
  FROM employees
  GROUP BY department_id
) d ON e.department_id = d.department_id AND e.salary =
d.avg_salary;
```

Q83 Keep only the first occurrence of each duplicate invoice number.

```
DELETE FROM invoices
WHERE invoice_id NOT IN (
  SELECT MIN(invoice_id)
  FROM invoices
  GROUP BY invoice_number
);
```

Window Functions in Practice

Running totals, moving averages, gaps-and-islands, and other analytics-style problems solved with OVER().

Q84 Calculate a running total of sales ordered by date.

```
SELECT sale_date, amount,
       SUM(amount) OVER (ORDER BY sale_date) AS
running_total
FROM sales;
```

Q85 Calculate a 3-order moving average of order amounts per customer.

```
SELECT customer_id, order_date, amount,
       AVG(amount) OVER (
         PARTITION BY customer_id
         ORDER BY order_date
         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
       ) AS moving_avg
FROM orders;
```

Q86 Find the difference between each order amount and the previous order amount for the same customer.

```
SELECT customer_id, order_date, amount,
       amount - LAG(amount) OVER (PARTITION BY
customer_id ORDER BY order_date) AS diff
FROM orders;
```

Window Functions in Practice

Running totals, moving averages, gaps-and-islands, and other analytics-style problems solved with OVER().

Q87 Find the first and last order date for each customer.

```
SELECT customer_id,
       MIN(order_date) AS first_order,
       MAX(order_date) AS last_order
FROM orders
GROUP BY customer_id;
```

Q88 Find the cumulative distribution of employee salaries.

```
SELECT name, salary,
       CUME_DIST() OVER (ORDER BY salary) AS cum_dist
FROM employees;
```

Q89 Calculate year-over-year revenue growth.

```
SELECT year, revenue,
       revenue - LAG(revenue) OVER (ORDER BY year) AS
yoy_growth
FROM annual_revenue;
```

Q90 Find the running count of orders placed by each customer.

```
SELECT customer_id, order_date,
       COUNT(*) OVER (PARTITION BY customer_id ORDER BY
order_date) AS order_sequence
FROM orders;
```

Window Functions in Practice

Running totals, moving averages, gaps-and-islands, and other analytics-style problems solved with OVER().

Q91 Find the next order amount for each customer using LEAD.

```
SELECT customer_id, order_date, amount,
       LEAD(amount) OVER (PARTITION BY customer_id ORDER
BY order_date) AS next_amount
FROM orders;
```

Q92 Find the highest salary so far for each employee ordered by date of raise.

```
SELECT employee_id, raise_date, salary,
       MAX(salary) OVER (PARTITION BY employee_id ORDER
BY raise_date) AS highest_so_far
FROM salary_history;
```

Q93 Calculate the percentage contribution of each product's sales to the total sales.

```
SELECT product_id, total_sales,
       ROUND(100.0 * total_sales / SUM(total_sales) OVER
(), 2) AS pct_of_total
FROM (
  SELECT product_id, SUM(quantity * price) AS
total_sales
  FROM order_items
  GROUP BY product_id
) t;
```

Window Functions in Practice

Running totals, moving averages, gaps-and-islands, and other analytics-style problems solved with OVER().

Q94 Find the 3rd highest salary per department using NTH_VALUE.

```
SELECT department_id,
       NTH_VALUE(salary, 3) OVER (
         PARTITION BY department_id ORDER BY salary DESC
         ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED
FOLLOWING
       ) AS third_highest
FROM employees;
```

Q95 Find the running maximum stock price per day.

```
SELECT trade_date, price,
       MAX(price) OVER (ORDER BY trade_date) AS
running_max
FROM stock_prices;
```

Q96 Identify the rank of each customer based on total spend, without gaps.

```
SELECT customer_id, total_spend,
       DENSE_RANK() OVER (ORDER BY total_spend DESC) AS
spend_rank
FROM (
  SELECT customer_id, SUM(amount) AS total_spend
  FROM orders
  GROUP BY customer_id
) t;
```

Window Functions in Practice

Running totals, moving averages, gaps-and-islands, and other analytics-style problems solved with OVER().

Q97 Find each employee's salary compared to the department average using window functions.

```
SELECT name, department_id, salary,
       salary - AVG(salary) OVER (PARTITION BY
department_id) AS diff_from_avg
FROM employees;
```

Q98 Calculate the running total of expenses reset at the start of every month.

```
SELECT expense_date, amount,
       SUM(amount) OVER (
         PARTITION BY DATE_TRUNC('month', expense_date)
         ORDER BY expense_date
       ) AS monthly_running_total
FROM expenses;
```

Q99 Find the previous and next salary of each employee ordered by salary.

```
SELECT name, salary,
       LAG(salary) OVER (ORDER BY salary) AS
prev_salary,
       LEAD(salary) OVER (ORDER BY salary) AS
next_salary
FROM employees;
```

Window Functions in Practice

Running totals, moving averages, gaps-and-islands, and other analytics-style problems solved with OVER().

Q100 Find the percentage change in monthly sales.

```
SELECT month, total_sales,
       ROUND(100.0 * (total_sales - LAG(total_sales)
OVER (ORDER BY month))
             / LAG(total_sales) OVER (ORDER BY month),
2) AS pct_change
FROM monthly_sales;
```

Q101 Identify the first purchase made by each customer using window functions.

```
SELECT *
FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY customer_id
ORDER BY order_date) AS rn
  FROM orders
) t
WHERE rn = 1;
```

Q102 Find a 7-day rolling average of daily website visits.

```
SELECT visit_date, visits,
       AVG(visits) OVER (
         ORDER BY visit_date
         ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
       ) AS rolling_7day_avg
FROM daily_visits;
```

Window Functions in Practice

Running totals, moving averages, gaps-and-islands, and other analytics-style problems solved with OVER().

Q103 Find the gap in days between consecutive orders for each customer.

```
SELECT customer_id, order_date,
       order_date - LAG(order_date) OVER (PARTITION BY
customer_id ORDER BY order_date) AS days_gap
FROM orders;
```

Q104 Rank products within each category by total revenue and keep only the top 2.

```
SELECT *
FROM (
  SELECT category_id, product_id, total_revenue,
         RANK() OVER (PARTITION BY category_id ORDER BY
total_revenue DESC) AS rnk
  FROM (
    SELECT p.category_id, oi.product_id, SUM(oi.quantity
* oi.price) AS total_revenue
    FROM order_items oi
    JOIN products p ON oi.product_id = p.product_id
    GROUP BY p.category_id, oi.product_id
  ) rev
) ranked
WHERE rnk <= 2;
```

Window Functions in Practice

Running totals, moving averages, gaps-and-islands, and other analytics-style problems solved with OVER().

Q105 Find the salary growth of each employee compared to their first recorded salary.

```
SELECT employee_id, raise_date, salary,
       salary - FIRST_VALUE(salary) OVER (
         PARTITION BY employee_id ORDER BY raise_date
       ) AS growth_since_start
FROM salary_history;
```

Q106 Find the median order amount using PERCENTILE_CONT.

```
SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY
amount) AS median_amount
FROM orders;
```

Q107 Find customers whose spending increased in every consecutive quarter.

```
SELECT customer_id
FROM (
  SELECT customer_id, quarter, amount,
         LAG(amount) OVER (PARTITION BY customer_id
ORDER BY quarter) AS prev_amount
  FROM quarterly_spend
) t
GROUP BY customer_id
HAVING SUM(CASE WHEN amount <= prev_amount THEN 1 ELSE 0
END) = 0;
```

Window Functions in Practice

Running totals, moving averages, gaps-and-islands, and other analytics-style problems solved with OVER().

Q108 Find the average time gap between orders for each customer.

```
SELECT customer_id, AVG(gap_days) AS avg_gap
FROM (
  SELECT customer_id,
         order_date - LAG(order_date) OVER (PARTITION BY
customer_id ORDER BY order_date) AS gap_days
  FROM orders
) t
GROUP BY customer_id;
```

Q109 Find the top performing salesperson for each month.

```
SELECT *
FROM (
  SELECT month, salesperson_id, total_sales,
         RANK() OVER (PARTITION BY month ORDER BY
total_sales DESC) AS rnk
  FROM monthly_sales_by_rep
) ranked
WHERE rnk = 1;
```

Window Functions in Practice

Running totals, moving averages, gaps-and-islands, and other analytics-style problems solved with OVER().

Q110 Find the cumulative number of new customers acquired each month.

```
SELECT signup_month, new_customers,
       SUM(new_customers) OVER (ORDER BY signup_month)
AS cumulative_customers
FROM (
  SELECT DATE_TRUNC('month', signup_date) AS
signup_month, COUNT(*) AS new_customers
  FROM customers
  GROUP BY DATE_TRUNC('month', signup_date)
) t;
```

Q111 Calculate the variance of each employee's salary from the company average using window functions.

```
SELECT name, salary,
       salary - AVG(salary) OVER () AS
variance_from_company_avg
FROM employees;
```

Q112 Find the last value of stock price in each trading day partitioned by stock symbol.

```
SELECT symbol, trade_date, price,
       LAST_VALUE(price) OVER (
         PARTITION BY symbol, trade_date ORDER BY
trade_time
         ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED
```

Date & Time Problems

Handling temporal data: streaks, gaps, age calculations and time-bucketed aggregations.

Q113 Find employees who joined in the last 6 months.

```
SELECT *
FROM employees
WHERE join_date >= CURRENT_DATE - INTERVAL '6 months';
```

Q114 Find customers who logged in on consecutive days (a login streak).

```
SELECT customer_id, login_date,
       login_date - ROW_NUMBER() OVER (PARTITION BY
customer_id ORDER BY login_date)::INT AS streak_group
FROM logins;
```

Q115 Find missing dates in a sequence of daily sales records.

```
SELECT d.calendar_date
FROM calendar d
LEFT JOIN sales s ON d.calendar_date = s.sale_date
WHERE s.sale_date IS NULL
  AND d.calendar_date BETWEEN (SELECT MIN(sale_date)
FROM sales)
                          AND (SELECT MAX(sale_date)
FROM sales);
```

Q116 Calculate each employee's age from their date of birth.

```
SELECT name, dob,
       EXTRACT(YEAR FROM AGE(CURRENT_DATE, dob)) AS age
```

Date & Time Problems

Handling temporal data: streaks, gaps, age calculations and time-bucketed aggregations.

Q117 Find orders placed on weekends.

```
SELECT *
FROM orders
WHERE EXTRACT(DOW FROM order_date) IN (0, 6);
```

Q118 Find the number of monthly active users.

```
SELECT DATE_TRUNC('month', activity_date) AS month,
COUNT(DISTINCT user_id) AS active_users
FROM user_activity
GROUP BY DATE_TRUNC('month', activity_date);
```

Q119 Find customers who have not placed an order in the last 90 days.

```
SELECT c.*
FROM customers c
WHERE c.customer_id NOT IN (
  SELECT customer_id FROM orders
  WHERE order_date >= CURRENT_DATE - INTERVAL '90 days'
);
```

Q120 Find the number of days between an employee's hire date and today.

```
SELECT name, join_date,
       CURRENT_DATE - join_date AS days_employed
FROM employees;
```

Date & Time Problems

Handling temporal data: streaks, gaps, age calculations and time-bucketed aggregations.

Q121 Find the busiest day of the week for orders.

```
SELECT TO_CHAR(order_date, 'Day') AS day_of_week,
COUNT(*) AS total_orders
FROM orders
GROUP BY TO_CHAR(order_date, 'Day')
ORDER BY total_orders DESC
LIMIT 1;
```

Q122 Find employees whose work anniversary is this month.

```
SELECT *
FROM employees
WHERE EXTRACT(MONTH FROM join_date) = EXTRACT(MONTH FROM
CURRENT_DATE);
```

Q123 Find the total sales for each quarter of the year.

```
SELECT EXTRACT(QUARTER FROM sale_date) AS quarter,
SUM(amount) AS total_sales
FROM sales
GROUP BY EXTRACT(QUARTER FROM sale_date);
```

Date & Time Problems

Handling temporal data: streaks, gaps, age calculations and time-bucketed aggregations.

Q124 Find the longest streak of consecutive days a user was active.

```
SELECT user_id, MAX(streak_length) AS longest_streak
FROM (
  SELECT user_id, COUNT(*) AS streak_length
  FROM (
    SELECT user_id, activity_date,
           activity_date - ROW_NUMBER() OVER (PARTITION
BY user_id ORDER BY activity_date)::INT AS grp
    FROM user_activity
  ) t
  GROUP BY user_id, grp
) streaks
GROUP BY user_id;
```

Q125 Find the average number of days between consecutive purchases for each customer.

```
SELECT customer_id, AVG(days_between) AS
avg_days_between
FROM (
  SELECT customer_id,
         order_date - LAG(order_date) OVER (PARTITION BY
customer_id ORDER BY order_date) AS days_between
  FROM orders
) t
GROUP BY customer_id;
```

Date & Time Problems

Handling temporal data: streaks, gaps, age calculations and time-bucketed aggregations.

Q126 Find all orders placed in the last 24 hours.

```
SELECT *
FROM orders
WHERE order_timestamp >= NOW() - INTERVAL '24 hours';
```

Q127 Find the month with the highest revenue.

```
SELECT DATE_TRUNC('month', sale_date) AS month,
SUM(amount) AS total_revenue
FROM sales
GROUP BY DATE_TRUNC('month', sale_date)
ORDER BY total_revenue DESC
LIMIT 1;
```

Q128 Find employees who have completed exactly 5 years of service.

```
SELECT *
FROM employees
WHERE join_date = CURRENT_DATE - INTERVAL '5 years';
```

Q129 Convert a UTC timestamp column to a specific time zone.

```
SELECT order_id, order_timestamp AT TIME ZONE 'UTC' AT
TIME ZONE 'Asia/Kolkata' AS local_time
FROM orders;
```

Date & Time Problems

Handling temporal data: streaks, gaps, age calculations and time-bucketed aggregations.

Q130 Find users who signed up but never logged in within 7 days.

```
SELECT u.*
FROM users u
LEFT JOIN logins l ON u.user_id = l.user_id
  AND l.login_date BETWEEN u.signup_date AND
u.signup_date + INTERVAL '7 days'
WHERE l.user_id IS NULL;
```

Q131 Find the number of new customers acquired each week.

```
SELECT DATE_TRUNC('week', signup_date) AS week, COUNT(*)
AS new_customers
FROM customers
GROUP BY DATE_TRUNC('week', signup_date);
```

Q132 Find orders that took more than 5 days to be shipped after being placed.

```
SELECT *
FROM orders
WHERE shipped_date - order_date > 5;
```

Aggregation edge cases that frequently trip up candidates in real interviews.

Q133 Find departments having more than 5 employees.

```
SELECT department_id, COUNT(*) AS num_employees
```

GROUP BY / HAVING Puzzles

> Aggregation edge cases that frequently trip up candidates in real interviews.

Q134 Find products ordered by more than 10 distinct customers.

```
SELECT oi.product_id, COUNT(DISTINCT o.customer_id) AS
distinct_customers
FROM order_items oi
JOIN orders o ON oi.order_id = o.order_id
GROUP BY oi.product_id
HAVING COUNT(DISTINCT o.customer_id) > 10;
```

Q135 Find departments whose average salary is above the company average.

```
SELECT department_id, AVG(salary) AS dept_avg
FROM employees
GROUP BY department_id
HAVING AVG(salary) > (SELECT AVG(salary) FROM
employees);
```

Q136 Find categories that have zero sales.

```
SELECT c.category_id, c.category_name
FROM categories c
LEFT JOIN products p ON c.category_id = p.category_id
LEFT JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY c.category_id, c.category_name
HAVING SUM(oi.quantity) IS NULL;
```

GROUP BY / HAVING Puzzles

> Aggregation edge cases that frequently trip up candidates in real interviews.

Q137 Find customers who have placed orders worth more than 5000 in total.

```
SELECT customer_id, SUM(amount) AS total_spent
FROM orders
GROUP BY customer_id
HAVING SUM(amount) > 5000;
```

Q138 Find job titles held by more than one employee.

```
SELECT job_title, COUNT(*) AS total
FROM employees
GROUP BY job_title
HAVING COUNT(*) > 1;
```

Q139 Find departments where the maximum salary is less than 60000.

```
SELECT department_id, MAX(salary) AS max_salary
FROM employees
GROUP BY department_id
HAVING MAX(salary) < 60000;
```

Q140 Find students who have failed more than 2 subjects.

```
SELECT student_id, COUNT(*) AS failed_subjects
FROM results
WHERE grade = 'F'
GROUP BY student_id
```

GROUP BY / HAVING Puzzles

> Aggregation edge cases that frequently trip up candidates in real interviews.

Q141 Find customers who ordered the same product more than 3 times.

```
SELECT customer_id, product_id, COUNT(*) AS
times_ordered
FROM order_items oi
JOIN orders o ON oi.order_id = o.order_id
GROUP BY customer_id, product_id
HAVING COUNT(*) > 3;
```

Q142 Find departments with a total headcount between 10 and 20.

```
SELECT department_id, COUNT(*) AS headcount
FROM employees
GROUP BY department_id
HAVING COUNT(*) BETWEEN 10 AND 20;
```

Q143 Find vendors supplying products with an average price above 1000.

```
SELECT vendor_id, AVG(price) AS avg_price
FROM products
GROUP BY vendor_id
HAVING AVG(price) > 1000;
```

GROUP BY / HAVING Puzzles

Aggregation edge cases that frequently trip up candidates in real interviews.

Q144 Find months where total sales dropped below 10000.

```
SELECT DATE_TRUNC('month', sale_date) AS month,
SUM(amount) AS total_sales
FROM sales
GROUP BY DATE_TRUNC('month', sale_date)
HAVING SUM(amount) < 10000;
```

Q145 Find employees grouped by job title having a combined salary above 500000.

```
SELECT job_title, SUM(salary) AS total_salary
FROM employees
GROUP BY job_title
HAVING SUM(salary) > 500000;
```

Q146 Find product categories with fewer than 5 products listed.

```
SELECT category_id, COUNT(*) AS total_products
FROM products
GROUP BY category_id
HAVING COUNT(*) < 5;
```

Q147 Find customers with orders in more than 3 different months.

```
SELECT customer_id, COUNT(DISTINCT DATE_TRUNC('month',
order_date)) AS active_months
FROM orders
```

GROUP BY / HAVING Puzzles

Aggregation edge cases that frequently trip up candidates in real interviews.

Q148 Find regions where total revenue exceeds 1 million.

```
SELECT region, SUM(amount) AS total_revenue
FROM sales
GROUP BY region
HAVING SUM(amount) > 1000000;
```

Q149 Find suppliers providing only a single product.

```
SELECT vendor_id, COUNT(*) AS total_products
FROM products
GROUP BY vendor_id
HAVING COUNT(*) = 1;
```

Q150 Find departments where the salary range (max-min) exceeds 50000.

```
SELECT department_id, MAX(salary) - MIN(salary) AS
salary_range
FROM employees
GROUP BY department_id
HAVING MAX(salary) - MIN(salary) > 50000;
```

Q151 Find customers whose average order value is below 100.

```
SELECT customer_id, AVG(amount) AS avg_order_value
FROM orders
GROUP BY customer_id
HAVING AVG(amount) < 100;
```

GROUP BY / HAVING Puzzles

Aggregation edge cases that frequently trip up candidates in real interviews.

Q152 Find employees grouped by department who collectively worked more than 1000 overtime hours.

```
SELECT department_id, SUM(overtime_hours) AS
total_overtime
FROM employees
GROUP BY department_id
HAVING SUM(overtime_hours) > 1000;
```

Correlated subqueries, recursive CTEs for hierarchies, and statistical queries like median and mode.

Q153 Find employees who earn more than their manager.

```
SELECT e.name AS Employee, e.salary, m.name AS Manager,
m.salary AS ManagerSalary
FROM employees e
JOIN employees m ON e.manager_id = m.id
WHERE e.salary > m.salary;
```

Subqueries & CTEs

Correlated subqueries, recursive CTEs for hierarchies, and statistical queries like median and mode.

Q154 Build the full organizational hierarchy using a recursive CTE.

```
WITH RECURSIVE org_chart AS (
  SELECT id, name, manager_id, 1 AS level
  FROM employees
  WHERE manager_id IS NULL
  UNION ALL
  SELECT e.id, e.name, e.manager_id, oc.level + 1
  FROM employees e
  JOIN org_chart oc ON e.manager_id = oc.id
)
SELECT * FROM org_chart
ORDER BY level;
```

Q155 Find the median salary in the employees table.

```
SELECT AVG(salary) AS median_salary
FROM (
  SELECT salary
  FROM employees
  ORDER BY salary
  LIMIT 2 - (SELECT COUNT(*) FROM employees) % 2
  OFFSET (SELECT (COUNT(*) - 1) / 2 FROM employees)
) AS median_subquery;
```

Subqueries & CTEs

Correlated subqueries, recursive CTEs for hierarchies, and statistical queries like median and mode.

Q156 Find the mode (most frequently occurring salary).

```
SELECT salary
FROM employees
GROUP BY salary
ORDER BY COUNT(*) DESC
LIMIT 1;
```

Q157 Find each employee's level/depth in the reporting hierarchy.

```
WITH RECURSIVE hierarchy AS (
  SELECT id, name, manager_id, 0 AS depth
  FROM employees
  WHERE manager_id IS NULL
  UNION ALL
  SELECT e.id, e.name, e.manager_id, h.depth + 1
  FROM employees e
  JOIN hierarchy h ON e.manager_id = h.id
)
SELECT * FROM hierarchy;
```

Subqueries & CTEs

Correlated subqueries, recursive CTEs for hierarchies, and statistical queries like median and mode.

Q158 Find customers whose total spend is greater than the average spend across all customers.

```
SELECT customer_id, total_spend
FROM (
  SELECT customer_id, SUM(amount) AS total_spend
  FROM orders
  GROUP BY customer_id
) c
WHERE total_spend > (
  SELECT AVG(total_spend) FROM (
    SELECT SUM(amount) AS total_spend FROM orders GROUP
BY customer_id
  ) avg_sub
);
```

Q159 Use a CTE to simplify a multi-step sales summary report.

```
WITH monthly_totals AS (
  SELECT DATE_TRUNC('month', sale_date) AS month,
SUM(amount) AS total
  FROM sales
  GROUP BY DATE_TRUNC('month', sale_date)
)
SELECT month, total,
       total - LAG(total) OVER (ORDER BY month) AS
month_over_month_change
FROM monthly_totals;
```

Subqueries & CTEs

Correlated subqueries, recursive CTEs for hierarchies, and statistical queries like median and mode.

Q160 Find products priced above the average price within their own category (correlated subquery).

```
SELECT p1.*
FROM products p1
WHERE p1.price > (
  SELECT AVG(p2.price)
  FROM products p2
  WHERE p2.category_id = p1.category_id
);
```

Q161 Find the total number of direct and indirect reports for each manager using a recursive CTE.

```
WITH RECURSIVE reports AS (
  SELECT id, manager_id FROM employees
  WHERE manager_id IS NOT NULL
  UNION ALL
  SELECT e.id, r.manager_id
  FROM employees e
  JOIN reports r ON e.manager_id = r.id
)
SELECT manager_id, COUNT(*) AS total_reports
FROM reports
GROUP BY manager_id;
```

Subqueries & CTEs

Correlated subqueries, recursive CTEs for hierarchies, and statistical queries like median and mode.

Q162 Find employees who do not appear in the project_assignments table using NOT EXISTS.

```
SELECT *
FROM employees e
WHERE NOT EXISTS (
  SELECT 1 FROM project_assignments pa WHERE
pa.employee_id = e.id
);
```

Q163 Find customers who bought a product also bought by customer 'Alice' (self-referencing subquery).

```
SELECT DISTINCT o2.customer_id
FROM orders o1
JOIN order_items oi1 ON o1.order_id = oi1.order_id
JOIN order_items oi2 ON oi1.product_id = oi2.product_id
JOIN orders o2 ON oi2.order_id = o2.order_id
WHERE o1.customer_id = (SELECT customer_id FROM
customers WHERE name = 'Alice')
  AND o2.customer_id <> o1.customer_id;
```

Subqueries & CTEs

Correlated subqueries, recursive CTEs for hierarchies, and statistical queries like median and mode.

Q164 Use a CTE to find the top spender in each region.

```
WITH regional_spend AS (
  SELECT region, customer_id, SUM(amount) AS
total_spend,
         RANK() OVER (PARTITION BY region ORDER BY
SUM(amount) DESC) AS rnk
  FROM orders o
  JOIN customers c ON o.customer_id = c.customer_id
  GROUP BY region, customer_id
)
SELECT * FROM regional_spend WHERE rnk = 1;
```

Q165 Find the percentage of employees earning above the median salary.

```
WITH median_calc AS (
  SELECT AVG(salary) AS median_salary
  FROM (
    SELECT salary FROM employees ORDER BY salary
    LIMIT 2 - (SELECT COUNT(*) FROM employees) % 2
    OFFSET (SELECT (COUNT(*) - 1) / 2 FROM employees)
  ) sub
)
SELECT 100.0 * COUNT(*) / (SELECT COUNT(*) FROM
employees) AS pct_above_median
FROM employees, median_calc
WHERE employees.salary > median_calc.median_salary;
```

Subqueries & CTEs

Correlated subqueries, recursive CTEs for hierarchies, and statistical queries like median and mode.

Q166 Find all subordinates (direct and indirect) of a given manager using a recursive CTE.

```
WITH RECURSIVE subordinates AS (
  SELECT id, name, manager_id
  FROM employees
  WHERE manager_id = 101
  UNION ALL
  SELECT e.id, e.name, e.manager_id
  FROM employees e
  JOIN subordinates s ON e.manager_id = s.id
)
SELECT * FROM subordinates;
```

Q167 Find departments where every employee earns above 50000 using NOT EXISTS.

```
SELECT DISTINCT department_id
FROM employees e1
WHERE NOT EXISTS (
  SELECT 1 FROM employees e2
  WHERE e2.department_id = e1.department_id AND
e2.salary <= 50000
);
```

Subqueries & CTEs

Correlated subqueries, recursive CTEs for hierarchies, and statistical queries like median and mode.

Q168 Find the cumulative count of orders using a CTE combined with a window function.

```
WITH daily_orders AS (
  SELECT order_date, COUNT(*) AS daily_count
  FROM orders
  GROUP BY order_date
)
SELECT order_date, daily_count,
       SUM(daily_count) OVER (ORDER BY order_date) AS
cumulative_orders
FROM daily_orders;
```

Q169 Find products that have never appeared together in the same order (using a self-join subquery).

```
SELECT p1.product_id AS product_a, p2.product_id AS
product_b
FROM products p1
CROSS JOIN products p2
WHERE p1.product_id < p2.product_id
  AND NOT EXISTS (
    SELECT 1 FROM order_items oi1
    JOIN order_items oi2 ON oi1.order_id = oi2.order_id
    WHERE oi1.product_id = p1.product_id AND
oi2.product_id = p2.product_id
  );
```

Subqueries & CTEs

Correlated subqueries, recursive CTEs for hierarchies, and statistical queries like median and mode.

Q170 Find the average tenure (in years) of employees per department using a CTE.

```
WITH tenure AS (
  SELECT department_id,
         EXTRACT(YEAR FROM AGE(CURRENT_DATE, join_date))
AS years_employed
  FROM employees
)
SELECT department_id, AVG(years_employed) AS avg_tenure
FROM tenure
GROUP BY department_id;
```

Q171 Find employees earning the maximum salary within their reporting chain (their own subtree) using recursion.

```
WITH RECURSIVE chain AS (
  SELECT id, name, manager_id, salary, id AS
root_manager
  FROM employees
  WHERE manager_id IS NULL
  UNION ALL
  SELECT e.id, e.name, e.manager_id, e.salary,
c.root_manager
  FROM employees e
  JOIN chain c ON e.manager_id = c.id
)
SELECT root_manager, MAX(salary) AS max_salary_in_tree
```

Subqueries & CTEs

Correlated subqueries, recursive CTEs for hierarchies, and statistical queries like median and mode.

Q172 Find customers whose most recent order amount is greater than their average order amount.

```
WITH customer_orders AS (
  SELECT customer_id, amount, order_date,
         AVG(amount) OVER (PARTITION BY customer_id) AS
avg_amount,
         ROW_NUMBER() OVER (PARTITION BY customer_id
ORDER BY order_date DESC) AS rn
  FROM orders
)
SELECT customer_id, amount, avg_amount
FROM customer_orders
WHERE rn = 1 AND amount > avg_amount;
```

Text manipulation: extracting, transforming, and matching string-based data.

Q173 Find employees with duplicate names (case-insensitive).

```
SELECT LOWER(name) AS name_lower, COUNT(*)
FROM employees
GROUP BY LOWER(name)
HAVING COUNT(*) > 1;
```

Q174 Extract the domain name from each employee's email address.

```
SELECT email, SUBSTRING(email FROM POSITION('@' IN
email) + 1) AS domain
```

String & Pattern Problems

Text manipulation: extracting, transforming, and matching string-based data.

Q175 Find employees whose name contains only alphabetic characters.

```
SELECT *
FROM employees
WHERE name ~ '^[A-Za-z ]+$';
```

Q176 Concatenate first and last name into a single full name column.

```
SELECT first_name || ' ' || last_name AS full_name
FROM employees;
```

Q177 Find the employee with the longest name.

```
SELECT name, LENGTH(name) AS name_length
FROM employees
ORDER BY LENGTH(name) DESC
LIMIT 1;
```

Q178 Find the employee with the shortest name.

```
SELECT name, LENGTH(name) AS name_length
FROM employees
ORDER BY LENGTH(name) ASC
LIMIT 1;
```

Q179 Convert all employee names to uppercase.

String & Pattern Problems

Text manipulation: extracting, transforming, and matching string-based data.

Q180 Find employees whose email does not contain '@'.

```
SELECT *
FROM employees
WHERE email NOT LIKE '%@%';
```

Q181 Find the count of vowels in each employee's name.

```
SELECT name,
       LENGTH(name) -
LENGTH(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(
         LOWER(name),
'a',''),'e',''),'i',''),'o',''),'u','')) AS vowel_count
FROM employees;
```

Q182 Find customers whose phone number does not match a 10-digit numeric pattern.

```
SELECT *
FROM customers
WHERE phone !~ '^[0-9]{10}$';
```

Q183 Trim leading and trailing spaces from product names.

```
SELECT TRIM(product_name) AS cleaned_name
FROM products;
```

String & Pattern Problems

Text manipulation: extracting, transforming, and matching string-based data.

Q184 Find the position of the substring 'mail' within each email address.

```
SELECT email, POSITION('mail' IN email) AS
position_in_string
FROM employees;
```

Q185 Replace all spaces in product names with underscores.

```
SELECT product_name, REPLACE(product_name, ' ', '_') AS
slug
FROM products;
```

Q186 Find employees whose name has more than one word.

```
SELECT *
FROM employees
WHERE name LIKE '% %';
```

Q187 Reverse each customer's name.

```
SELECT name, REVERSE(name) AS reversed_name
FROM customers;
```

Real Interview Favorites / Hard

Tricky, high-frequency interview puzzles: gaps-and-islands, pivoting, and complex existence checks.

Q188 Find the 3 consecutive numbers that are missing from an ID sequence.

```
SELECT id + 1 AS missing_start
FROM sequence_table s1
WHERE NOT EXISTS (SELECT 1 FROM sequence_table s2 WHERE
s2.id = s1.id + 1)
  AND NOT EXISTS (SELECT 1 FROM sequence_table s3 WHERE
s3.id = s1.id + 2)
  AND NOT EXISTS (SELECT 1 FROM sequence_table s4 WHERE
s4.id = s1.id + 3);
```

Q189 Pivot monthly sales rows into columns (one column per month).

```
SELECT product_id,
       SUM(CASE WHEN month = 'Jan' THEN amount END) AS
Jan,
       SUM(CASE WHEN month = 'Feb' THEN amount END) AS
Feb,
       SUM(CASE WHEN month = 'Mar' THEN amount END) AS
Mar
FROM monthly_sales
GROUP BY product_id;
```

Real Interview Favorites / Hard

Tricky, high-frequency interview puzzles: gaps-and-islands, pivoting, and complex existence checks.

Q190 Unpivot columns (Jan, Feb, Mar) back into rows.

```
SELECT product_id, 'Jan' AS month, jan_sales AS amount
FROM sales_wide
UNION ALL
SELECT product_id, 'Feb', feb_sales FROM sales_wide
UNION ALL
SELECT product_id, 'Mar', mar_sales FROM sales_wide;
```

Q191 Find employees who never took a single day of leave.

```
SELECT e.*
FROM employees e
LEFT JOIN leave_records l ON e.id = l.employee_id
WHERE l.employee_id IS NULL;
```

Real Interview Favorites / Hard

Tricky, high-frequency interview puzzles: gaps-and-islands, pivoting, and complex existence checks.

Q192 Find the top N products per category using window functions.

```
SELECT *
FROM (
  SELECT p.category_id, p.product_name, s.total_sales,
         ROW_NUMBER() OVER (PARTITION BY p.category_id
ORDER BY s.total_sales DESC) AS rnk
  FROM products p
  JOIN (
    SELECT product_id, SUM(quantity * price) AS
total_sales
    FROM order_items
    GROUP BY product_id
  ) s ON p.product_id = s.product_id
) ranked
WHERE rnk <= 3;
```

Q193 Find customers who bought every single product in a given category.

```
SELECT o.customer_id
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
WHERE p.category_id = 5
GROUP BY o.customer_id
HAVING COUNT(DISTINCT p.product_id) = (
```

Real Interview Favorites / Hard

Tricky, high-frequency interview puzzles: gaps-and-islands, pivoting, and complex existence checks.

Q194 Find consecutive groups of dates (islands) where a machine was continuously active.

```
SELECT machine_id, MIN(active_date) AS island_start,
MAX(active_date) AS island_end
FROM (
  SELECT machine_id, active_date,
         active_date - ROW_NUMBER() OVER (PARTITION BY
machine_id ORDER BY active_date)::INT AS grp
  FROM machine_activity
) t
GROUP BY machine_id, grp;
```

Q195 Find the longest consecutive run of login days for any single user.

```
SELECT user_id, COUNT(*) AS streak_length
FROM (
  SELECT user_id, login_date,
         login_date - ROW_NUMBER() OVER (PARTITION BY
user_id ORDER BY login_date)::INT AS grp
  FROM logins
) t
GROUP BY user_id, grp
ORDER BY streak_length DESC
LIMIT 1;
```

Real Interview Favorites / Hard

Tricky, high-frequency interview puzzles: gaps-and-islands, pivoting, and complex existence checks.

Q196 Transpose a table of exam scores so each subject becomes its own column per student.

```
SELECT student_id,
       MAX(CASE WHEN subject = 'Math' THEN score END) AS
Math,
       MAX(CASE WHEN subject = 'Science' THEN score END)
AS Science,
       MAX(CASE WHEN subject = 'English' THEN score END)
AS English
FROM exam_scores
GROUP BY student_id;
```

Q197 Find employees who have an identical set of skills as another employee.

```
SELECT e1.employee_id, e2.employee_id AS
matching_employee
FROM employee_skills e1
JOIN employee_skills e2
  ON e1.skill_id = e2.skill_id AND e1.employee_id <>
e2.employee_id
GROUP BY e1.employee_id, e2.employee_id
HAVING COUNT(*) = (SELECT COUNT(*) FROM employee_skills
WHERE employee_id = e1.employee_id)
   AND COUNT(*) = (SELECT COUNT(*) FROM employee_skills
WHERE employee_id = e2.employee_id);
```

Real Interview Favorites / Hard

Tricky, high-frequency interview puzzles: gaps-and-islands, pivoting, and complex existence checks.

Q198 Find the three most recently registered customers in each country.

```
SELECT *
FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY country
ORDER BY signup_date DESC) AS rnk
  FROM customers
) ranked
WHERE rnk <= 3;
```

Q199 Find products whose sales dropped for two consecutive months in a row.

```
SELECT product_id, month
FROM (
  SELECT product_id, month, total_sales,
         LAG(total_sales, 1) OVER (PARTITION BY
product_id ORDER BY month) AS prev1,
         LAG(total_sales, 2) OVER (PARTITION BY
product_id ORDER BY month) AS prev2
  FROM monthly_product_sales
) t
WHERE total_sales < prev1 AND prev1 < prev2;
```

Real Interview Favorites / Hard

Tricky, high-frequency interview puzzles: gaps-and-islands, pivoting, and complex existence checks.

Q200 Find the employee-manager pair with the largest salary gap.

```
SELECT e.name AS employee, m.name AS manager, (m.salary
- e.salary) AS salary_gap
FROM employees e
JOIN employees m ON e.manager_id = m.id
ORDER BY salary_gap DESC
LIMIT 1;
```